Scalable Community Detection Using Quantum Hamiltonian Descent and QUBO Formulation

Jinglei Cheng[§*], Ruilin Zhou[†*], Yuhang Gan[†], Chen Qian[†], Junyu Liu[§]
[§]University of Pittsburgh [†]University of California, Santa Cruz
[*]Authors contributed equally to this research. Corresponding to: jic373@pitt.edu, junyuliu@pitt.edu

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133263

***Abstract*—We present a quantum-inspired algorithm that utilizes Quantum Hamiltonian Descent (QHD) for efficient community detection. Our approach reformulates the community detection task as a Quadratic Unconstrained Binary Optimization (QUBO) problem, and QHD is deployed to identify optimal community structures. We implement a multi-level algorithm that iteratively refines community assignments by alternating between QUBO problem setup and QHD-based optimization. Benchmarking shows our method achieves up to 5.49% better modularity scores while requiring less computational time compared to classical optimization approaches. This work demonstrates the potential of hybrid quantum-inspired solutions for advancing community detection in large-scale graph data.**

***Index Terms*—Quantum Hamiltonian Descent, QUBO, Community Detection**

I. Introduction

Quantum computing with the principles of quantum mechanics can process information in different ways from classical computers. This is because qubits can exist in superposition, and therefore can represent multiple states simultaneously [1]. This capability, combined with entanglement and interference, allows quantum algorithms to solve certain classes of problems, such as integer factorization and combinatorial optimization, more efficiently than classical methods. Therefore, quantum computing has lead to advancements in cryptography [2], optimization [3], machine learning [4], and materials science [5]. In parallel, quantum-inspired algorithms bring similar quantum concepts to classical computing, with principles such as tunneling, adiabatic transitions, and Hamiltonian dynamics to address complex problems without requiring quantum hardware. These approaches have demonstrated enhanced performance in areas like traffic routing and combinatorial optimization, where classical systems emulate quantum behaviors to achieve good efficiency [6].

Community detection is an important problem in network analysis, and it's essential for interpreting the structural and functional organization of complex systems [7]. An example is given in Figure 1. The overall goal is to identify communities—groups of nodes with denser connections among themselves than with the rest of the network [8]. In large-scale graphs, which are common in various real-world applications such as social networks, biological systems, and communication networks, effective community detection provides valuable insights into network behavior and properties, and can support applications like anomaly detection, network optimization, and information dissemination [9]. The impact of community detection is deep across many domains. In social networks, it enables the identification of groups with shared interests or behaviors, which can enhance recommendation systems and targeted marketing strategies [10], [11]. In biological networks, community detection reveals functional modules or protein complexes, and contributes to a better understanding of cellular processes and disease mechanisms [12], [13]. Additionally, fields like communication networks, financial systems, and transportation networks takes community detection to optimize efficiency, bolster security, and improve resilience against failures or attacks [14], [15].

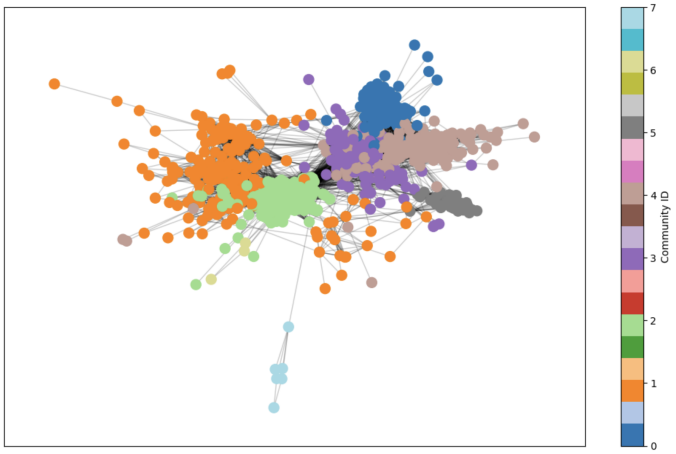

Fig. 1. Visualization of community structure in a complex network. The network consists of nodes grouped into communities, each represented by a different color.

Detecting communities in large-scale graphs presents substantial computational challenges. Traditional algorithms often struggle with scalability, resulting in increased processing times and high resource consumption as network size grows [16]. Furthermore, maintaining the accuracy and quality of detected communities becomes more complex with larger datasets due to the intricate network structures and the presence of noise [10]. These challenges call for the development of more scalable algorithms capable of handling large-scale graphs without compromising performance or accuracy [17].

Quantum-inspired algorithms inherit conceptions from quantum computing to enhance classical computational methods [18]–[22]. Quantum Hamiltonian Descent (QHD) [23], [24] is an algorithm designed to efficiently solve optimization problems by leveraging quantum tunneling effects to escape local minima, and therefore enhances performance in non-convex optimization scenarios. By simulating the behavior of quantum systems, Quantum Hamiltonian Descent can navigate complex energy landscapes to find optimal solutions more effectively than some traditional optimization techniques [25]. This makes it a promising approach for tackling the computational demands of large-scale community detection, where the formulation is not always convex.

Our approach involves formulating the community detection problem as a QUBO model as shown in Figure 2. The QUBO formulation is a versatile framework for representing combinatorial optimization problems where the objective is to minimize a quadratic function of binary variables [26]. By expressing community detection in this form, we can apply optimization algorithms like Quantum Hamilto-

nian Descent directly. In this way, the challenge of running graph algorithms on GPUs can be addressed by transforming them into a QUBO formulation, which allows for the deployment of GPU-accelerated algorithms. Quantum Hamiltonian Descent is particularly effective for solving QUBO models due to its ability to efficiently explore huge and complex solution spaces. By mimicking the dynamics of quantum systems, it can avoid becoming trapped in local minima and converge rapidly to high-quality solutions. This efficiency is essential for large-scale problems where traditional optimization methods may be too slow or require excessive computational resources.

To further improve efficiency and solution quality, we have developed a multi-level algorithm that iteratively refines community assignments. This algorithm alternates between QUBO formulation and Quantum Hamiltonian Descent solving at different levels of graph granularity. Starting from a coarse representation of the graph, it progressively refines the community structure by incorporating more detailed information in each iteration. This hierarchical approach helps in escaping local optima and enhances the overall accuracy of the community detection process.

Our method has been benchmarked against GUROBI [27], which is a leading commercial solver renowned for its optimization capabilities. On smaller problem instances, our algorithm matches GUROBI's performance, demonstrating its effectiveness and reliability. Notably, for larger problems exceeding 1,000 variables, our approach surpasses GUROBI. This performance advantage is crucial for practical applications involving large-scale graphs where computational resources and time are significant constraints. Recognizing the computational intensity of large-scale community detection, we have implemented our algorithm on multi-GPU architectures. GPUs offer massive parallel processing power, which accelerates the Quantum Hamiltonian Descent process and handles the extensive computations required for large graphs. Our implementation demonstrates excellent scalability and makes it feasible to analyze large networks in reasonable time budget. This efficient use of high-performance computing resources indicates the practicality of our approach for real-world applications.

Our contributions are:

- Developed a novel quantum-inspired community detection algorithm utilizing QHD optimization
- Formulated community detection as a QUBO problem with an iterative multi-level refinement approach
- Demonstrated competitive performance with GUROBI solver on small instances and superior scalability for problems exceeding 1,000 variables

The remainder of this paper is organized as follows. Section II provides the necessary background on community detection, QUBO problems, and quantum-inspired optimization techniques. In Section III, we present the formal mathematical framework of our approach, detailing the QUBO formulation. Section IV describes our implementation, including the multi-level algorithm design and QHD algorithm. We present comprehensive experimental results and performance analysis in Section V. Finally, Section VI concludes the paper with a summary of our contributions and directions for future research.

II. Background

A. Quantum Hamiltonian Descent

QHD introduces a new quantum counterpart to classical gradient descent methods, derived through path integral quantization of the continuous-time limit of gradient descent algorithms. Unlike conventional approaches that only quantize specific components of classical algorithms, QHD quantizes the entire dynamical system, resulting in a quantum evolution governed by a Hamiltonian $\hat{H}(t) = e^{\phi_t}(-\frac{1}{2}\Delta) + e^{\chi_t} f(x)$, where e^{ϕ_t} and e^{χ_t} are damping parameters controlling system energy flow. This formulation allows QHD to take advantages of quantum tunneling effects to escape local minima by considering all possible paths at the same time, including those prohibited in classical mechanics. The algorithm exhibits three distinct phases - kinetic, global search, and descent - and has demonstrated superior performance over both classical gradient-based methods and quantum adiabatic algorithms in solving non-convex optimization problems.

B. Community Detection

Community Detection (CD), also known as graph clustering or graph partition, is one fundamental task in graph theory and network science with the main goal of identifying groups of nodes within a graph that are more densely connected to each other than the rest. Applications of community detection span various fields, such as social network analysis [28], biology [29], computer networks, and recommendation systems [30]. Based on the definition of the nodes, applications, and the definition of a good partition. Various methods have been proposed, such as the modularity-based method [31], which aims to maximize modularity, which is a measure of the strength of the division of a network into communities, spectral clustering, which involves the use the eigenvalues and eigenvectors of matrices like the Laplacian derived from the graph to identify community structures based on the network's spectral properties and hierarchical clustering which use methods such as divisive clustering to build a hierarchy of communities either by progressively merging nodes (bottom-up) or recursively splitting larger communities (top-down) based on similarity measures.

III. Formalization

In this section, we introduce how we transform the CD problem into QUBO form and the main considerations of our formulations.

A. Modularity

Formally, the community detection problem with the goal of maximizing modularity can be formulated as follows: Given an undirected graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges, the task of community detection is to partition the vertices into k non-empty communities such that nodes within the same community are more densely connected while nodes in different communities are sparsely connected. One commonly used metric to measure how well the graph is partitioned is 'modularity' which is defined by:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{d_i d_j}{2m} \right) \delta(c_i, c_j) \tag{1}$$

where A_{ij} is the element of the adjacency matrix, d_i and d_j are the degrees of nodes i and j. And $\delta(c_i c_j)$ is the Kronecker delta function that equals 1 if nodes i and j are in the same community and 0 otherwise.

Various heuristics have been proposed to address the community detection problem. Cut-based methods aim to minimize inter-community edges while maintaining balanced partitions [32]. Modularity-based methods focus on maximizing the density of intra-community edges relative to a null model [33]. Another approach involves using quantum annealing; however, this method introduces additional overhead due to the need to embed the problem instance onto the quantum annealer [34].

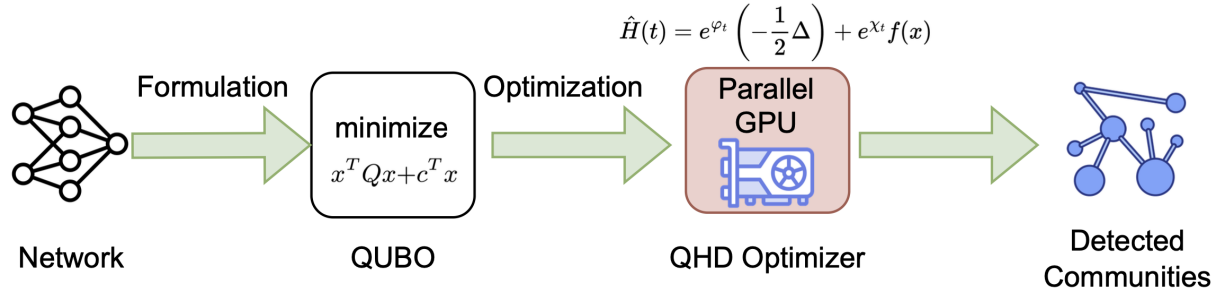

Fig. 2. Overview of our quantum-inspired community detection approach. The input network is reformulated as a QUBO optimization problem, which is then solved using QHD accelerated by parallel GPU computation. The QHD optimizer uses quantum-inspired dynamics to efficiently identify lowest cost function values in the complex landscape, and delivers superior scalability for instances with thousands of nodes.

B. QUBO Formulation for Community Detection

Inspired by previous classical and quantum methods, we propose a quantum-inspired method that takes Quantum Hamiltonian Descent to replace the original compute-intensive components of classical algorithms. This method first transforms the CD problem into a QUBO formulation, which is the first step for the use of both classical and quantum-inspired solvers that can be executed on GPUs without the need for direct embedding onto quantum hardware. The specific formalization is as follows.

Algorithm 1 QUBO Construction for Community Detection

Input:
n: number of nodes
k: number of communities
B: modularity matrix
$G(V, E)$: input graph
w_1, w_2, w_3: penalty weights
Output: QUBO matrix Q and linear terms b
Initialize $Q \in \mathbb{R}^{nk \times nk}, b \in \mathbb{R}^{nk}$ to zero
function IDX(i, c) **return** $i \cdot k + c$
end function
// Modularity term
$Q_{\text{idx}(i,c),\text{idx}(j,c)} \leftarrow -w_1 B_{ij}$
// Assignment constraints
$Q_{\text{idx}(i,c_1),\text{idx}(i,c_2)} \leftarrow 2w_2$
$b_{\text{idx}(i,c)} \leftarrow -w_2$
// Cut penalty
$i_c \leftarrow \text{idx}(u, c), j_c \leftarrow \text{idx}(v, c)$
$Q_{i_c,j_c} \leftarrow -2w_3$

1) Direct QUBO Formulation for Small Networks: To adapt the modularity optimization problem for QUBO solvers, we construct a QUBO objective function that encapsulates both the modularity measure and the necessary constraints for valid community assignments. Let binary variables $x_{i,c} \in \{0, 1\}$, where:

$$x_{i,c} = \begin{cases} 1 & \text{if vertex } i \text{ is assigned to community } c, \\ 0 & \text{otherwise.} \end{cases}$$

Assume a maximum of k communities, with $c \in \{1, 2, \ldots, k\}$. The modularity contribution can be expressed using the binary variables as:

$$Q_M = \frac{1}{2m} \sum_{i,j \in V} \left(A_{ij} - \frac{d_i d_j}{2m} \right) \sum_{c=1}^{k} x_{i,c} x_{j,c}. \tag{2}$$

. Different from the previous formalization, which only focused on maximizing modularity, we also added other terms. To ensure that each vertex is assigned to exactly one community, we impose the following constraint:

$$Q_A = \lambda_A \sum_{i \in V} \left(1 - \sum_{c=1}^{k} x_{i,c} \right)^2, \tag{3}$$

where λ_A is a penalty coefficient that enforces the constraints, and expanding the squared term ensures that derivation from the constraint incur a quadratic penalty. To prevent trivial solution where all vertices are assigned to a single community or most vertices are assigned to several communities and aim to provide a balance size of communities, we also introduce the constraints on the size of communities:

$$Q_S = \lambda_S \sum_{c=1}^{k} \left(\sum_{i \in V} x_{i,c} - \frac{n}{k} \right)^2, \tag{4}$$

where λ_S is the penalty coefficient for balancing community sizes and $\frac{n}{k}$ represents the desired average community size. Based on these, the complete QUBO objective function to be maximized can be formed as

$$Q = -Q_M + Q_A + Q_S, \tag{5}$$

where $-Q_M$ corresponds to the maximization of the modularity, Q_A enforces the constraint that each vertex will be in exactly one community, and Q_S enforces a balanced constraint, which is also a consideration in most CD formalization. In general, our formalization gives comprehensive objective functions that take multiple aspects into consideration instead of solely maximizing the modularity metric. More details are shown in Algorithm III-B.

2) Scale to Larger Networks: While previous direct QUBO formalization is effective for small to medium-sized networks($|V| \leq 1000$), larger networks require more scalable approaches. To address this and inspired by classical methods [35], we employ a hierarchical methodology that maintains solution quality while reducing computational complexity. More specifically, our method includes the following steps: Coarsening, initial partition, uncoarsening, and refinement. More details are shown in Algorithm III-B2.

Coarsening Phase: We iteratively reduce the graph size by aggregating vertices into super-nodes, thereby preserving the community structure at multiple scales. At each coarsening step, a heavy-edge matching strategy is utilized to select vertex pairs with strong connectivity:

$$w(e) = \alpha \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|} + \beta \frac{A_{uv}}{\max_{e \in E} A_{uv}}, \tag{6}$$

Algorithm 2 Multilevel Community Detection

Input:
$G_0(V_0, E_0)$: input graph
k: number of communities
θ: coarsening threshold
Output: Community assignments $P : V_0 \to 1, \ldots, k$
function COARSEN(G_i)
    Match nodes in G_i to form super-nodes
    Combine matched nodes to create G_{i+1} **return** G_{i+1}
end function
function PROJECT(P_{i+1}, G_i)
    Map community assignments from level $i+1$ to i **return** P_i
end function
// Coarsening Phase
$i \leftarrow 0$
while $|V_i| > \theta$ **do**
    $G_{i+1} \leftarrow$ COARSEN(G_i)
    $i \leftarrow i + 1$
end while
$m \leftarrow i$ // Final level
// Initial Solution
$P_m \leftarrow$ SOLVEBASE(G_m, k)
// Uncoarsening Phase
for $i = m - 1$ **downto** 0 **do**
    $P_i \leftarrow$ PROJECT(P_{i+1}, G_i)
    $P_i \leftarrow$ REFINE(P_i, G_i, k)
end for
return P_0

where $N(u)$ denotes the neighborhood of vertex $u.\alpha$, and β are weighting coefficients that balance the contribution of neighborhood overlap and edge weight. This coarsening phase reduces the number of vertices while preserving the essential community structure.

Initial Partition: Once the graph is sufficiently coarsened to a manageable size, we apply the direct QUBO formulation (5) to partition the coarsest graph. This step provides an initial community assignment that captures the high-level structure of the original graph.

Uncoarsening and Refinement: After initial partitioning is found in the coarsest graph, the partitioning is then projected back to the original graph through the following steps:

1) **Projection**: Starting from the coarsest level, iteratively map the community from each coarsened graph G_i to the next finer graph G_{i-1}. This is based on the super-node correspondence established during the coarsening phase which can effectively transfer the high-level community structure to more detailed graph levels.
2) **Refinement**: At each level, we optimize the community assignments by iteratively reassigning nodes to communities that result in the highest gain in modularity. This involves evaluating potential moves for each node and updating its community assignment if the move improves the overall modularity. The refinement process continues until certain iterations of the threshold are met or no further improvement is achieved.

IV. IMPLEMENTATION

A. QHD on QUBO

QHDOPT [25] implements Quantum Hamiltonian Descent for solving nonlinear optimization problems. Its key innovation is implementing this optimization via quantum evolution governed by the time-dependent Schrödinger equation:

$$i\frac{\partial}{\partial t}\Psi(t,x) = \left(e^{\phi_t}\left(-\frac{1}{2}\Delta\right) + e^{\chi_t}f(x)\right)\Psi(t,x)$$

where Δ is the Laplacian operator and e^{ϕ_t}, e^{χ_t} control the energy distribution of the quantum system.

A significant computational advantage of this approach is that it requires only matrix multiplication operations, avoiding the need to solve linear systems (Ax=b). This makes the method particularly amenable to modern hardware acceleration. For practical implementation, QHDOPT employs a discretization and embedding strategy where the continuous Hamiltonian is discretized into an N^n-dimensional operator:

$$\hat{H}(t) = e^{\phi_t}\left(-\frac{1}{2}L_d\right) + e^{\chi_t}F_d$$

where L_d represents the discretized Laplacian and F_d encodes the objective function.

The method's matrix-multiplication-based formulation opens up possibilities for various compression techniques that can transform large sparse problems into smaller, denser ones. This includes techniques like tensor network compression [36], hierarchical matrix approximations [37], and randomized sketching methods [38]. Any additional constraints created during problem transformation can be efficiently handled through penalty-based methods, avoiding the need for complex constraint satisfaction algorithms.

When dealing with QUBO problems, QHDOPT can be efficiently accelerated using GPUs through careful implementation of the quantum Hamiltonian dynamics. The key is mapping the QUBO problem $\min_{x\in\{0,1\}^n} x^TQx+b^Tx$ to QHD's Hamiltonian framework in a way that exploits massive parallelism. Sparse operations can be particularly accelerated using specialized GPU packages like cuSPARSE [39] or MAGMA [40], which provide optimized implementations for sparse matrix operations. The time evolution can be implemented using GPU-optimized linear algebra operations, with the Laplacian operator Δ computed using parallel finite difference schemes and the potential term $e^{\chi_t}f(x)$ evaluated using batched matrix operations.

By leveraging frameworks like PyTorch [41] or JAX [42] that provide automatic differentiation and GPU acceleration, the quantum dynamics can be simulated efficiently - the gradient computations for the Hamiltonian evolution can be parallelized across the quantum state dimensions, and the time evolution steps can be batched for exploring multiple initial conditions simultaneously. The classical refinement step in QHDOPT can utilize GPU-accelerated optimizers to efficiently project solutions back to binary values. This hybrid quantum-classical approach combines quantum effects with massive parallel computing capabilities, making it particularly effective for large-scale optimization problems where traditional methods might struggle.

B. Multi-level Community Detection

This multilevel community detection algorithm is designed to efficiently handle large-scale graphs by employing a hierarchical approach. The algorithm operates in three main phases: First, in the coarsening phase, it repeatedly combines nodes to create a hierarchy of progressively smaller graphs while preserving the community structure. Once the graph is sufficiently small, it solves the community detection problem directly on this coarsened graph. Finally, in the uncoarsening phase, it progressively maps the solution

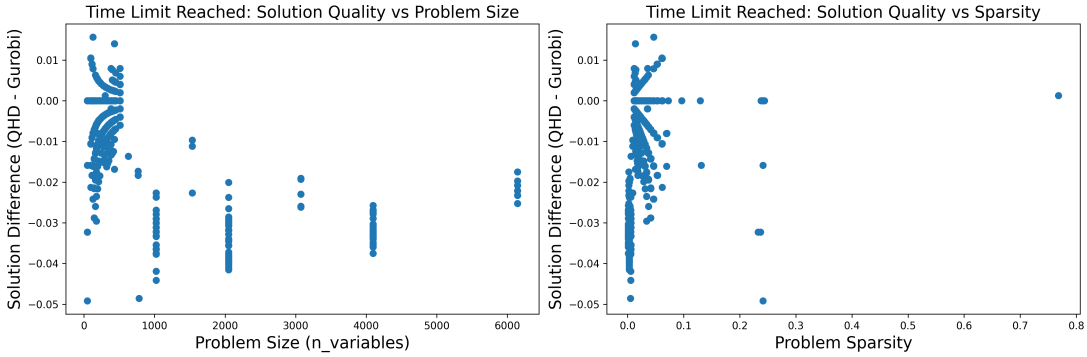

Fig. 3. Solution Quality Comparison When GUROBI Hits Time Limit: For 739 instances where GUROBI exceeded time limits (predominantly in larger problems), QHD found better solutions in 71.4% of cases. For larger-scale problems, QHD demonstrated superior performance by finding better solutions than GUROBI within the same time constraints. This advantage stems from QHD's inherent parallel processing capabilities, which can be further enhanced through additional GPU resources and optimized sparse matrix operations, suggesting even greater potential for scaling to larger problem instances.

back to finer levels while refining the communities at each step. This multilevel approach allows the algorithm to maintain both computational efficiency and solution quality by working with smaller problems during the initial solution while preserving the ability to make fine-grained adjustments during refinement.

V. Evaluation

A. Evaluation Setting

The experimental evaluation was conducted on a system running Debian Linux with kernel version 5.10.0-22-amd64. The computing platform features a dual 16-core CPU (32 cores total). The server is equipped with 251 GB of RAM. Implementation of QHD on QUBO is running with four NVIDIA A5000 GPUs and the comparison with GUROBI is conducted with CPUs as mentioned.

B. QUBO Solver with GUROBI and QHD

Comparing the computational performance between GUROBI and QHD presents unique challenges due to their different control parameters. While GUROBI can be configured with node exploration limits and termination times, QHD allows adjustment of sample sizes and iteration counts. To establish comparisons, we adopted a time-based benchmarking approach: first measuring QHD's execution time, then allocating GUROBI the same as its time limit. This methodology is justified because superior performance can be demonstrated either through faster execution at equal solution quality or through better solutions within equal time constraints. Our experiments focused on the latter approach, comparing solution quality between the two solvers under equivalent time constraints. The results demonstrate that QHD achieves higher solution quality while GUROBI cannot finish with status as OPTIMAL within the allocated time. It's worth noting that QHD's performance can be further enhanced with additional computational resources due to its inherent massive parallelism capabilities. The results in Figure **??** and Figure 3 demonstrate robust performance patterns across a substantial dataset of 938 instances. In the 199 cases where GUROBI found proven optimal solutions (typically smaller problems with mean size of 54 variables), QHD matched these optimal solutions in 75.4% of cases. More importantly, in the larger set of 739 instances where GUROBI hit its time limit (predominantly larger problems with mean size of 614 variables), QHD demonstrated superior performance by finding better solutions in 71.4% of cases and matching GUROBI's solutions in another 17.2%. While the time-limited instances show lower sparsity (mean 0.028 vs 0.157), this is primarily a characteristic of larger problem instances rather than a direct driver of performance difference. The results, based on this comprehensive dataset, strongly indicate QHD's effectiveness for larger-scale optimization problems where traditional solvers face computational limitations, regardless of sparsity.

C. Evaluation on Small Size Networks

Our experimental evaluation compared the Quantum Hamiltonian Descent approach with the GUROBI-based modularity optimization across a diverse set of network instances. As shown in Table I, the test bed has 10 networks ranging from a graph with (52 nodes and 146 edges) to an instance of (1,034 nodes, 26,749 edges), with edge densities from 3.4% to 15.2%. As shown in Fig. 5, the QHD-based method demonstrated robust performance, achieving higher modularity scores in 8 out of 10 instances compared to the GUROBI implementation. The average modularity difference of 0.0029 in favor of QHD suggests that quantum-inspired optimization can effectively match or slightly exceed traditional exact optimization approaches. This robust performance, where QHD achieves higher modularity scores in 8 out of 10 test instances while using only 20% of GUROBI's computational time with four GPUs, demonstrates its practical advantages over classical modularity optimization methods, with the potential for even greater efficiency gains through increased GPU parallelization.

TABLE I
Instance Properties and Modularity Scores

Instance	Nodes	Edges	Density %	GUROBI	QHD
0	333	2,519	4.56	0.4523	**0.4610**
107	1,034	26,749	5.01	**0.5290**	0.5241
348	224	3,192	12.78	0.3055	**0.3063**
414	150	1,693	15.15	0.5438	**0.5438**
686	168	1,656	11.80	0.3347	**0.3347**
698	61	270	14.75	0.5369	**0.5369**
1684	786	14,024	4.55	0.5528	**0.5640**
1912	747	30,025	10.78	0.5167	**0.5239**
3437	534	4,813	3.38	0.6724	**0.6784**
3980	52	146	11.01	**0.4619**	0.4619

D. Evaluation on Large Size Networks

Table II and Figure 6 of GUROBI and QHD performance across different networks reveal a significant density-dependent pattern. In the Facebook network (density 0.0108), QHD achieves a notable 5.49% higher modularity score (0.7512 ± 0.0258) compared to

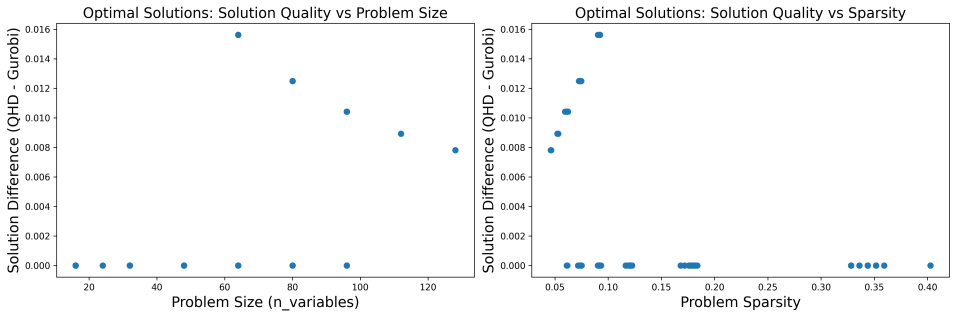

Fig. 4. Solution Quality Comparison When GUROBI Reaches Optimality: Among 199 instances where GUROBI found optimal solutions (predominantly in smaller problems), QHD achieved identical solutions in 75.4% of cases. In cases where QHD did not match GUROBI's optimal solutions (24.6% of optimally solved instances), the objective value differences remained within a 1.6% relative gap, primarily occurring in smaller instances where the absolute differences in objective values were minimal.

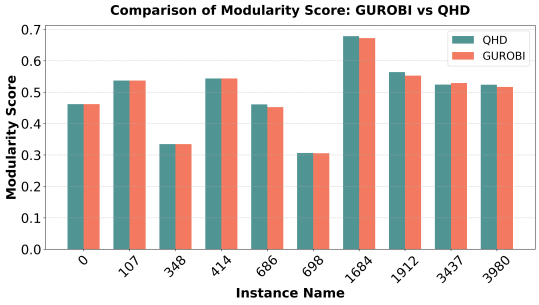

Fig. 5. Performance comparison between QHD and GUROBI on network instances ranging from 52 to 1,034 nodes. QHD achieves superior modularity scores in 80% of test cases with an average improvement of 0.0029, while requiring only 20% of GUROBI's computational time using four GPUs. Results demonstrate QHD's effectiveness across different network densities (3.4%-15.2%).

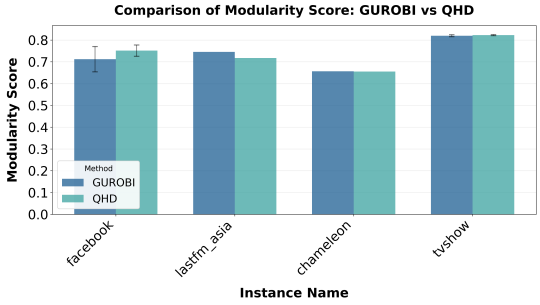

Fig. 6. The performance advantage varies with network density, from QHD's 5.49% improvement on Facebook (density 0.0108) to GUROBI's 3.79% advantage on sparse LastFM (density 0.0010). Both methods show comparable performance on medium-density networks.

GUROBI (0.7121 ± 0.0579), while also demonstrating better stability with lower variance. The densest network, Chameleon (density 0.0121), shows nearly identical performance between the methods, with only a 0.19% difference favoring GUROBI. The TVShow network, despite its lower density (0.0023), exhibits near performance from both methods, with QHD showing a marginal advantage of 0.33% (QHD: 0.8223 ± 0.0025, GUROBI: 0.8196 ± 0.0044) and both methods maintaining very low variance. Notably, in the sparsest network, LastFM (density 0.0010), GUROBI outperforms QHD by 3.79%. These results indicate that QHD's advantages are most pronounced in moderately dense networks. And GUROBI maintains good performance across network types, particularly in sparser networks, which is attributed to its branch-and-cut algorithm.

TABLE II
COMPARISON OF GRAPH PROPERTIES AND MODULARITY SCORES

Instance	Nodes	Edges	Density %	GUROBI	QHD
facebook	4,039	88,234	1.08	0.7121	**0.7512**
lastfm_asia	7,626	27,807	0.10	**0.7455**	0.7172
musae_chameleon	2,279	31,372	1.21	**0.6567**	0.6554
tvshow	3,894	17,240	0.23	0.8196	**0.8223**

VI. CONCLUSION

This work demonstrates the effectiveness of quantum-inspired optimization for community detection through our QHD-based approach. By reformulating community detection as a QUBO problem and leveraging GPU-accelerated quantum-inspired optimization, we achieve comparable or superior performance to traditional methods while reducing computational time. Our experimental results show strong performance on moderately dense networks, with up to 5.49% improvement in modularity scores and enhanced stability compared to GUROBI. The implementation of multi-GPU acceleration suggests further scalability potential for larger networks. These findings highlight the promising intersection of quantum-inspired algorithms and high-performance computing for complex network analysis tasks. Future work could explore extending this approach to other graph optimization problems and investigating additional acceleration techniques for handling ultra-large-scale networks. Better designed algorithms to formulate graphs into denser matrices can reduce the number of variables in QUBO, and combination with high-performance sparsity computation will also be helpful. Our results demonstrate that quantum-inspired methods, combined with modern computing architectures, offer a practical pathway for advancing the capabilities of community detection in real-world network analysis applications.

VII. ACKOWLEDGEMENT

JL is supported in part by the University of Pittsburgh, School of Computing and Information, Department of Computer Science, Pitt Cyber, PQI Community Collaboration Awards. And by NASA under award number 80NSSC25M7057. This research used resources of the Oak Ridge Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC05-00OR22725. RL is supported by NSF Grants 2322919, 2420632, 2426031, 2426940, 2114113, and DOE Grant DE-SC0022069. We thank Yuxiang Peng from University of Maryland and Jiaqi Leng from UC Berkely for their helpful and insightul discussions.

REFERENCES

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th ed. Cambridge, UK: Cambridge University Press, 2010.

[2] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*. IEEE, 1994, pp. 124–134.

[3] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv preprint arXiv:1411.4028*, 2014. [Online]. Available: https://arxiv.org/abs/1411.4028

[4] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, pp. 195–202, 2017.

[5] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, pp. 242–246, 2017.

[6] A. Laio and M. Parrinello, "Escaping free-energy minima," *Proceedings of the national academy of sciences*, vol. 99, no. 20, pp. 12 562–12 566, 2002.

[7] S. Fortunato, "Community detection in graphs," *Physics Reports*, vol. 486, no. 3–5, pp. 75–174, 2010.

[8] M. Girvan and M. E. J. Newman, "Community structure in social and biological networks," *Proceedings of the National Academy of Sciences*, vol. 99, no. 12, pp. 7821–7826, 2002.

[9] M. E. J. Newman, *Networks: An Introduction*. Oxford, UK: Oxford University Press, 2010.

[10] S. Fortunato and D. Hric, "Community detection in networks: A user guide," *Physics Reports*, vol. 659, pp. 1–44, 2016.

[11] F. D. Malliaros and M. Vazirgiannis, "Clustering and community detection in directed networks: A survey," *Physics Reports*, vol. 533, no. 4, pp. 95–142, 2013.

[12] E. Ravasz, A. L. Somera, D. A. Mongru, Z. N. Oltvai, and A.-L. Barabasi, "Hierarchical organization of modularity in metabolic networks," *Science*, vol. 297, no. 5586, pp. 1551–1555, 2002.

[13] A.-L. Barabasi, N. Gulbahce, and J. Loscalzo, "Network medicine: A network-based approach to human disease," *Nature Reviews Genetics*, vol. 12, no. 1, pp. 56–68, 2011.

[14] M. Newman, *Networks*. Oxford, UK: Oxford University Press, 2018.

[15] S. Boccaletti, V. Latora, Y. Moreno, M. Chavez, and D.-U. Hwang, "Complex networks: Structure and dynamics," *Physics Reports*, vol. 424, no. 4–5, pp. 175–308, 2006.

[16] G. Rossetti and R. Cazabet, "Community discovery in dynamic networks: A survey," *ACM Computing Surveys*, vol. 51, no. 2, pp. 1–37, 2018.

[17] E. Abbe, "Community detection and stochastic block models: Recent developments," *Journal of Machine Learning Research*, vol. 18, no. 1, pp. 6446–6531, 2017.

[18] J. M. Arrazola, A. Delgado, B. R. Bardhan, and S. Lloyd, "Quantum-inspired algorithms in practice," *Quantum*, vol. 4, p. 307, 2020.

[19] N. Chepurko, K. Clarkson, L. Horesh, H. Lin, and D. Woodruff, "Quantum-inspired algorithms from randomized numerical linear algebra," *Proceedings of the 39th International Conference on Machine Learning*, vol. 162, pp. 3879–3900, 2022.

[20] C. Shao and A. Montanaro, "Faster quantum-inspired algorithms for solving linear systems," *arXiv preprint arXiv:2103.10309*, 2021.

[21] V. Yelleti and R. Krishna, "Quantum-inspired evolutionary algorithms for feature subset selection: A comprehensive survey," *arXiv preprint arXiv:2407.17946*, 2023.

[22] H. Okawa, Q.-G. Zeng, X.-Z. Tao, and M.-H. Yung, "Quantum-annealing-inspired algorithms for track reconstruction at high-energy colliders," *arXiv preprint arXiv:2402.14718*, 2024.

[23] J. Leng, E. Hickman, J. Li, and X. Wu, "Quantum hamiltonian descent," *arXiv preprint arXiv:2303.01471*, 2023.

[24] J. Leng, J. Li, Y. Peng, and X. Wu, "Expanding hardware-efficiently manipulable hilbert space via hamiltonian embedding," *arXiv preprint arXiv:2401.08550*, 2024.

[25] S. Kushnir, J. Leng, Y. Peng, L. Fan, and X. Wu, "Qhdopt: A software for nonlinear optimization with quantum hamiltonian descent," *arXiv preprint arXiv:2409.03121*, 2024.

[26] G. Kochenberger, J.-K. Hao, F. Glover, M. Lewis, Z. Lü, and H. Wang, "The unconstrained binary quadratic programming problem: A survey," *Journal of Combinatorial Optimization*, vol. 28, no. 1, pp. 58–81, 2014.

[27] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2024. [Online]. Available: https://www.gurobi.com

[28] M. Azaouzi, D. Rhouma, and L. Ben Romdhane, "Community detection in large-scale social networks: state-of-the-art and future directions," *Social Network Analysis and Mining*, vol. 9, no. 1, p. 23, 2019.

[29] F. R. Khawaja, Z. Zhang, Y. Memon, and A. Ullah, "Exploring community detection methods and their diverse applications in complex networks: a comprehensive review," *Social Network Analysis and Mining*, vol. 14, no. 1, p. 115, 2024.

[30] F. Gasparetti, G. Sansonetti, and A. Micarelli, "Community detection in social recommender systems: a survey," *Applied Intelligence*, vol. 51, pp. 3975–3995, 2021.

[31] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Physical Review E*, vol. 69, no. 2, p. 026113, 2004.

[32] H. Shin, J. Park, and D. Kang, "A graph-cut-based approach to community detection in networks," *Applied Sciences*, vol. 12, no. 12, p. 6218, 2022.

[33] M. Rosvall, J.-C. Delvenne, M. T. Schaub, and R. Lambiotte, "Different approaches to community detection," *Advances in network clustering and blockmodeling*, pp. 105–119, 2019.

[34] M. Fernández-Campoamor, C. O'Meara, G. Cortiana, V. Peric, and J. Bernabé-Moreno, "Community detection in electrical grids using quantum annealing," *arXiv preprint arXiv:2112.08300*, 2021.

[35] G. Karypis and V. Kumar, *METIS: A Software Package for Partitioning Unstructured Graphs, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices*, University of Minnesota, Minneapolis, MN, 1998, version 5.1.0. [Online]. Available: http://glaros.dtc.umn.edu/gkhome/metis/metis/overview

[36] A. Cichocki, R. Zdunek, A. H. Phan, and S.-i. Amari, "Tensor networks for dimensionality reduction and large-scale optimization: Part 1 low-rank tensor decompositions," *Foundations and Trends® in Machine Learning*, vol. 9, no. 4-5, pp. 249–429, 2016.

[37] W. Hackbusch, *Hierarchical Matrices: Algorithms and Analysis*. Springer, 2015.

[38] D. P. Woodruff, "Sketching as a tool for numerical linear algebra," *Foundations and Trends® in Theoretical Computer Science*, vol. 10, no. 1–2, pp. 1–157, 2014.

[39] NVIDIA Corporation, *cuSPARSE Library*, 2024, version 12.6. [Online]. Available: https://docs.nvidia.com/cuda/cusparse/index.html

[40] U. o. T. Innovative Computing Laboratory, *MAGMA Library*, 2024, version 2.8.0. [Online]. Available: https://icl.utk.edu/magma/

[41] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[42] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "Jax: composable transformations of python+numpy programs," 2018. [Online]. Available: http://github.com/google/jax

VQT-CiM: Accelerating Vector Quantization Enhanced Transformer with Ferroelectric Compute-in-Memory

Xuchu Huang[1], Haonan Du[1], Min Zhou[1], Zheyu Yan[1*], Cheng Zhuo[1*] and Xunzhao Yin[1,2*]
[1]Zhejiang University, Hangzhou, China; [2]Key Laboratory of CS&AUS of Zhejiang Province, Hangzhou, China;
[*]Corresponding author email: {zyan2, czhuo, xzyin1}@zju.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133264

ABSTRACT

Transformer models have achieved state-of-the-art performance in various natural language processing (NLP) and computer vision (CV) tasks. To meet their substantial computational demands, the compute-in-memory (CiM) architectures, which alleviate the memory wall problem and enable efficient vector-matrix multiplication (VMM), have been adopted for transformer accelerators. However, the dynamic VMM involved in the attention mechanism, which necessitates runtime write operations, presents significant challenges for non-volatile memory (NVM)-based CiM designs. High write overhead, complex compute-write-compute (CWC) dependencies, and limited endurance reduce their effectiveness. In this paper, we propose VQT-CiM, a ferroelectric FET (FeFET)-based CiM design that accelerates vector quantization (VQ) enhanced transformers by eliminating the runtime write operations. VQT-CiM quantizes keys and values in self-attention to convert dynamic VMMs in inner-product and weighted-sum into static VMMs with the codebooks, enabling efficient calculations with CiM crossbars. However, directly applying VQ hinders the accuracy of transformer model due to its limited representation capability. To address this, we introduce a vector quantization scheme that integrates residual VQ (RVQ) and product VQ (PVQ) for enhanced representation space. We present an efficient hardware implementation for the proposed VQT-CiM with optimized dataflow in RVQ, which incorporates the FeFET-based CiM crossbars and peripheral digital circuits. Evaluation results suggest that VQT-CiM achieves the 3.54× and 4.53× improvements in energy efficiency and throughput, respectively, compared to state-of-the-art NVM-based CiM transformer designs.

1 INTRODUCTION

Attention-based transformers [1] have demonstrated remarkable performance in natural language processing (NLP) [2, 3] and computer vision (CV) [4] tasks. Leveraging the global perceptive field enabled by self-attention mechanism, transformers excel at capturing long-range dependencies and complex relationships in data, leading to outstanding modeling capabilities. However, the extensive matrix multiplications involved in transformers, coupled with the quadratic computational complexity with respect to the sequence length, result in significant overhead in data communication and computation. Consequently, the specialized hardware accelerators for transformers have become increasingly essential.

Compute-in-memory (CiM) has emerged as a promising architecture, which alleviates the memory wall problem by integrating memory and computation. Leveraging the high parallelism of crossbar structures, CiM performs vector-matrix multiplication (VMM) with high efficiency, making it well-suited for deep learning tasks. Several studies have proposed CiM implementations as neural network accelerators, primarily categorized into non-volatile memory (NVM)-based [5, 6] and SRAM-based [7, 8] designs.

Different from Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), which rely on static VMM between dynamic inputs and static weights, transformers utilize dynamic VMM in self-attention calculations. The inner-product and weighted-sum in self-attention involve VMM between two dynamically generated matrices, leading to runtime write operations to CiM crossbars and complex compute-write-compute (CWC) dependencies. This is especially challenging for NVM devices, which suffer from high writing overhead and limited endurance. To mitigate this issue, researchers have designed SRAM-based [9, 10] and SRAM-NVM hybrid architectures [11], utilizing SRAM's low write overhead and sufficient endurance. However, the 6T-SRAM cell, combined with additional logical gates for multiplication and area-consuming adder trees for accumulation, results in lower storage density compared to NVM-based designs. Moreover, the volatile nature of SRAM limits its use in embedded devices, where data loss occurs when power turns off. The hybrid architectures also introduce design complexity and manufacturing challenges due to the heterogeneous integration, limiting their widespread adoption. Thus, designing fully static solutions that support NVM-based CiM implementations for transformers is of great importance.

A viable solution is to adopt vector quantization (VQ) technique into self-attention mechanism [12–14]. By substituting keys with their closest matches from a pre-trained codebook, Transformer-vq [14] achieves self-attention with linear complexity. Following this approach, the runtime write operations in the inner-product can be avoided, while the dynamically generated values still need to be written into CiM crossbars for weighted-sum. In this paper, we propose VQT-CiM, a ferroelectric FET (FeFET)-based CiM design for VQ-enhanced transformers, which fully eliminates the runtime write operations in self-attention. VQT-CiM employs VQ to both keys and values, converting dynamic VMMs in inner-product and weighted-sum into static VMMs. These static VMMs, which handle similarity measurement with the codebooks and post-substitution self-attention, are efficiently implemented using FeFET crossbars.

One of the main challenges with this approach is model performance degradation due to the limited representation capability of the quantized keys and values. Simply increasing the codebook size does not effectively mitigate this issue and instead raises the associated hardware overhead. To handle this problem, VQT-CiM integrates residual VQ (RVQ) [15] and product VQ (PVQ) [16]. RVQ utilizes a multi-stage VQ approach which refines the residuals between the input vectors and the quantized results, while PVQ employs multi-space VQ for more accurate approximations.

However, RVQ's iterative process, which involves residual calculations and similarity checks at each stage, presents challenges for hardware implementation due to its sequential nature. To address this, VQT-CiM optimizes the dataflow to enable parallelization of RVQ process. By introducing an additional look-up table (LUT) that stores pre-calculated similarity scores between codes, the codebooks across all stages can be accessed simultaneously for parallel

similarity measurements with the input vector. We present the hardware design based on these algorithm optimizations, which includes the FeFET-based CiM crossbars for static VMMs and digital circuits for additional VQ operations, resulting in an efficient and high-performance solution for VQ-enhanced transformers.

The contributions of this work are summarized as follows:

- We eliminate runtime write operations in CiM implementations for transformers by converting the dynamic VMMs into static, codebook-based VMMs through vector quantized keys and values in self-attention.
- We introduce a vector quantization scheme that significantly enhances VQ's representation capability by integrating RVQ and PVQ.
- We propose an efficient hardware design for VQT-CiM with optimized dataflow in RVQ, which comprises FeFET-based CiM crossbars and peripheral digital circuits.
- Evaluation results show that the VQT-CiM achieves 3.54× and 4.53× improvements in energy efficiency and throughput, compared to the state-of-the-art NVM-based CiM transformer designs.

2 BACKGROUND

In this section, we discuss basics of self-attention and related CiM implementations, followed by preliminaries of VQ, FeFET and CiM.

2.1 Self-attention Basics and Existing CiM Implementations

Fig. 1 illustrates the self-attention mechanism [1] in transformers, which consists of three stages: feature transformation, inner-product and weighted-sum. Feature transformation projects the input sequence into queries (Q), keys (K) and values (V) through three distinct weight matrices, W_Q, W_K and W_V. As described in Eq. (1), in the inner-product stage, the attention scores are calculated using $Q \cdot K^T$, capturing the relevance between input tokens. These scores are then normalized using scaled softmax, where d is the feature dimension. In the weighted-sum stage, the attention scores are applied to V to generate the output. Tokens with higher attention scores contribute more to the output Z.

$$Z = Softmax(\frac{Q \cdot K^T}{\sqrt{d}}) \cdot V \tag{1}$$

Several NVM-based CiM implementations for transformers have been proposed [17], [18], [19]. ReTransformer [17] optimizes the dataflow to reduce dynamic write operations to the CiM crossbar. ReBERT [18] mitigates data hazards by applying local attention, focusing on adjacent tokens in the attention score computation. CPSAA [19] achieves enhanced sparsity by computing the attention mask at runtime, merging W_Q and W_K to facilitate mask generation. Despite these advancements, NVM-based CiM designs still face challenges due to the dynamic VMM in self-attention. The high overhead of write operations, coupled with the complicated CWC dependencies, hinder their performance. Furthermore, the endurance issue of NVM devices is often neglected.

2.2 Vector Quantization Basics

Vector Quantization (VQ) [12, 13] maps continuous vectors into discrete codes from a predefined codebook C. As described in Eq. (2), VQ selects code with the highest similarity to represent the input vector. In this paper, we employ the hardware-friendly inner product for similarity measurement, rather than Euclidean distance.

$$z = \underset{i}{argmin}\, ||x - C[i]||^2 \approx \underset{i}{argmax}\, x \cdot C[i] \tag{2}$$

To integrate VQ into DNN training, the straight-through estimator (STE) [13, 20] is employed to address the non-differentiability of the maximum operation in VQ. During forward propagation, the quantized code $C[z]$ is utilized, while in backward propagation, the original gradient is applied, as detailed in Eq. (3), where SG indicates the stop gradient operation.

$$\overline{x} = x + SG(C[z] - x) \tag{3}$$

The codebook is updated using the exponential moving average (EMA) method [21], as described in Eq. (4). In each batch, codes are updated based on the assigned vectors. Here, $n[i]$ represents the number of vectors assigned to code i, and $x[j]$s are the corresponding input vectors. The parameter γ controls the decay rate, determining the update speed of the codebook.

$$\begin{cases} N[i]^{(t)} = N[i]^{(t-1)} * \gamma + n[i]^{(t)} * (1-\gamma) \\ M[i]^{(t)} = M[i]^{(t-1)} * \gamma + \sum_{j}^{n[i]^{(t)}} x[j]^{(t)} * (1-\gamma) \\ C[i]^{(t)} = \frac{M[i]^{(t)}}{N[i]^{(t)}} \end{cases} \tag{4}$$

2.3 FeFET and CiM Basics

FeFET has emerged as a competitive candidate in NVM devices for CiM implementations, owing to its CMOS compatibility, high ON/OFF ratio, and voltage-driven mechanism [22–24]. As illustrated in Fig. 2(a), the HfO_2 layer serves as ferroelectric dielectric

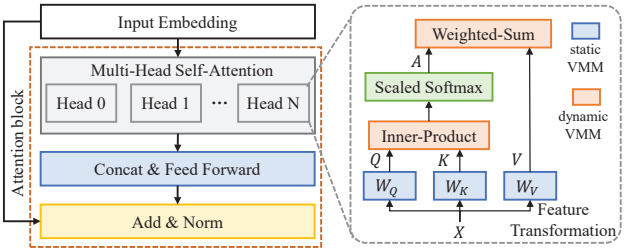

Figure 1: Self-attention block in transformer.

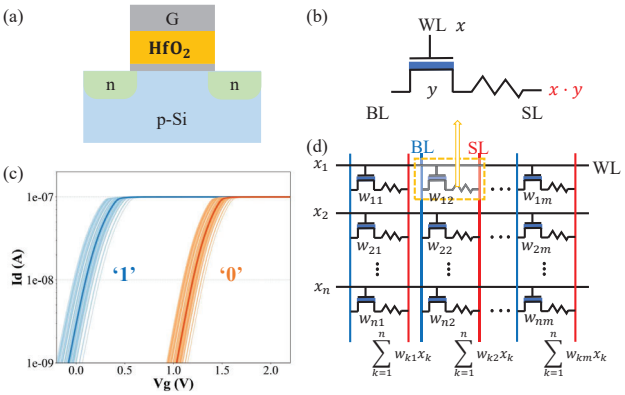

Figure 2: (a) FeFET structure. (b) The 1FeFET1R structure for CiM cell. (c) I_D-V_G curves with the series-connected resistor. (d) 1FeFET1R-based CiM crossbar.

in the gate, allowing for adjustable threshold voltage V_{th}. Different gate pulses are employed to alter the polarization states of HfO_2 to store the data. Specifically, positive gate pulses correspond to Vth_{low} ('1'), while negative gate pulses correspond to Vth_{high} ('0'). Fig. 2(b) shows the 1FeFET-1R structure [25–27], which incorporates a series-connected resistor to mitigate current variation. Simulation results for the 1T1R cell, based on the experimental calibrated FeFET compact model [28], are presented in Fig. 2(c).

For CiM calculations, as described in Fig. 2(b), when applying an input voltage to WL, the read current reflects the multiplication result of input and stored weight. In the crossbar structure of CiM, as depicted in Fig. 2(d), the read currents in SL are naturally summed according to Kirchhoff's Law, enabling efficient multiply-accumulate (MAC) operations [29, 30].

3 ALGORITHM OPTIMIZATION

In this section, we present the algorithm optimization in VQT-CiM. We introduce how VQ is integrated into self-attention in Sec. 3.1, and methods to enhance VQ's representation capacity in Sec. 3.2.

3.1 Self-Attention with VQ

Fig. 3(a) illustrates the dataflow of the self-attention mechanism. The dynamic VMMs in both inner-product and weighted-sum, necessitate runtime write operations for keys and values, leading to significant overhead and complicated CWC dependencies. To address this issue, we propose an algorithm optimization that approximates keys in inner-product and values in weighed-sum using VQ. This approach converts the dynamic VMMs into hardware-friendly static VMMs, as described in Fig. 3(b) and Fig. 3(c).

Key-VQ: The Key-VQ process starts with initializing a codebook $C_k \in R^{C,D}$, where C represents the number of the codes in the codebook, and D denotes the feature dimension. As described in Eq. (5), the similarity scores between keys and codes are calculated, resulting in $D_k \in R^{N,C}$, where N is the number of tokens. The one-hot matrix $\Delta_k \in R^{N,C}$ is then generated through a row-wise maximum operation, setting the position of the highest score in each row to 1, with the rest to 0. The quantized key, $\overline{K}$, which selects the corresponding code based on the one-hot matrix, substitutes K for inner-product calculation, as shown in Eq. (6) and Fig. 3(b).

$$D_k = K \cdot C_k^T \overset{one-hot}{\longrightarrow} \Delta_k \in R^{N,C} \tag{5}$$

$$K \overset{VQ}{\approx} \overline{K} = \Delta_k \cdot C_k \in R^{N,D} \tag{6}$$

To align with the fixed codebooks in CiM crossbars, we reformulate the inner-product calculation as shown in Eq. (7), which computes attention scores between Q and codebook C_k instead of the original $\overline{K}$. As illustrated in Fig. 3(c), the optimized dataflow achieves reduced latency in self-attention by enabling parallel computation of D_q and D_k. This approach also converts the original dynamic VMM in inner-product into two static VMMs with codebook C_k, which can be efficiently implemented with FeFET crossbars. The additional VQ operations, including the row-wise maximum for code selection and reshaping $Q \cdot C_k^T$ based on the one-hot matrix, are handled by a digital circuit with minimal hardware overhead.

$$\begin{aligned} A = Softmax(S) &\approx Softmax(Q \cdot \overline{K}^T) \\ = Softmax(Q \cdot (C_k^T \cdot \Delta_k^T)) &= Softmax((Q \cdot C_k^T) \cdot \Delta_k^T) \\ = Softmax((Q \cdot C_k^T) \cdot Onehot(K \cdot C_k^T)^T) & \end{aligned} \tag{7}$$

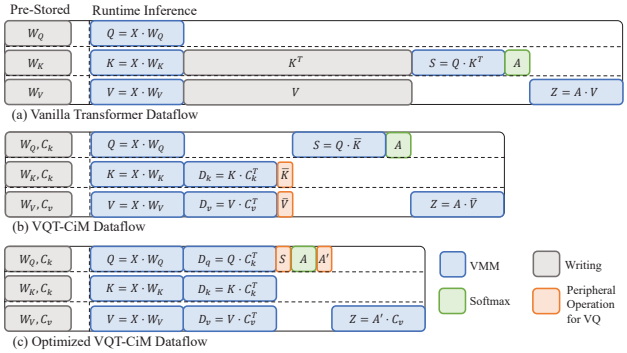

Figure 3: (a) Dataflow for vanilla self-attention. (b) Dataflow of VQT-CiM. (c) Optimzied dataflow of VQT-CiM.

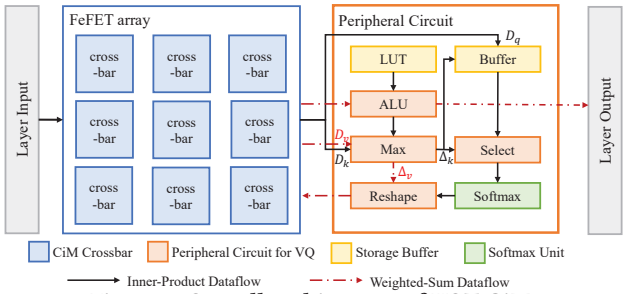

Figure 4: Overall architecture of VQT-CiM.

Value-VQ: Similarly, the Value-VQ process utilizes the codebook $C_v \in R^{C,D}$ for quantization. Eq. (8) describes the similarity scores calculation and row-wise maximum operation for code selection. Using the one-hot matrix Δ_v, V is approximated by $\overline{V}$ in the weighted-sum calculation by picking the most similar code from the codebook, as shown in Eq. (9) and Fig. 3(b).

$$D_v = V \cdot C_v^T \overset{one-hot}{\longrightarrow} \Delta_v \in R^{N,C} \tag{8}$$

$$V \overset{VQ}{\approx} \overline{V} = \Delta_v \cdot C_v \in R^{N,D} \tag{9}$$

To facilitate CiM implementations, the weighted-sum with Value-VQ is reformulated, as shown in Eq. (10) and Fig. 3(c), converting the dynamic VMM into two static VMMs based on the codebook C_v. It's important to note that the attention scores A, obtained from the inner-product step, are firstly transformed using the one-hot matrix Δ_v before participating in the static VMM. As with the hardware implementations for Key-VQ, the static VMMs are executed through FeFET crossbars, while the extra operations introduced by VQ are handled by digital circuits with negligible overhead.

$$\begin{aligned} Z = A \cdot V &\approx A \cdot (\Delta_v \cdot C_v) = (A \cdot \Delta_v) \cdot C_v \\ &= A' \cdot C_v = (A \cdot Onehot(V \cdot C_v^T)) \cdot C_v \end{aligned} \tag{10}$$

3.2 VQ Representation Capacity Enhancement

To mitigate the approximation error between the original vector and its quantized result, RVQ and PVQ are utilized to enhance the codebook's representation capacity. We employ RVQ and PVQ to both keys and values, as will be discussed in Sec. 4.2 and Sec. 4.3.

Residual Vector Quantization (RVQ): RVQ applies a multi-stage VQ process, where each stage sequentially refines the approximation by quantizing the residuals from the previous stage. Distinct codebooks are assigned to each VQ layer, denoted as $\{C_1, C_2, \ldots, C_R\}$,

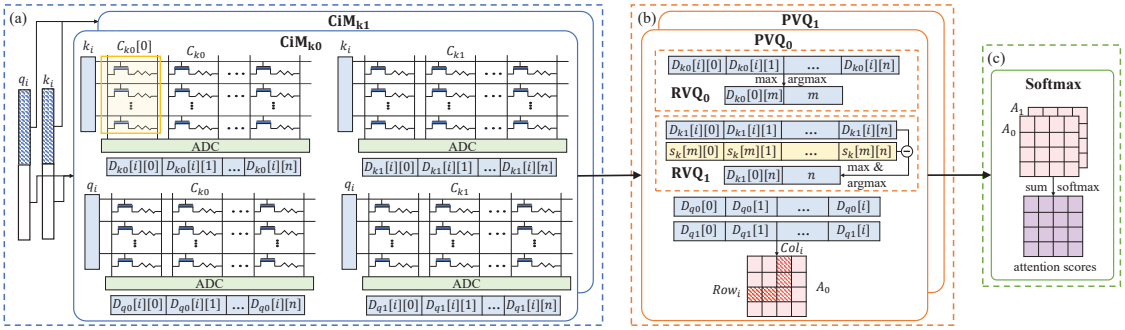

Figure 5: Hardware implementation of inner-product part in self-attention. (a) Static VMM with the codebook stored in FeFET crossbars. (b) Peripheral operations required by VQ. (c) Softmax for attention score normalization.

where R represents the number of layers. In the first layer, the input vector x is quantized using the codebook C_1, producing the code index z_1. The residual, defined as the difference between the input vector x and the quantized result $C_1[z_1]$, is then passed to the next VQ layer for further quantization, as described in Eq. (11). This process continues iteratively across all VQ layers, progressively reducing the residual error. As expressed in Eq. (12), the input vector x is ultimately approximated as the sum of the quantized vectors from all R VQ layers. Through this approach, RVQ substantially improves flexibility and expressiveness by combining codes across different codebooks. With C codes in each codebook, RVQ achieves the representation capacity of $O(C^R)$.

$$\begin{cases} z_1 = \underset{i}{argmax}\, x \cdot C_1[i] \\ z_k = \underset{i}{argmax}(x - \sum_{j=1}^{k-1} C_j[z_j]) \cdot C_k[i], 1 < k \le R \end{cases} \tag{11}$$

$$x \approx C_1[z_1] + C_2[z_2] + ... + C_R[z_R] \tag{12}$$

Product Vector Quantization (PVQ): PVQ divides the input vector into several shorter subvectors to achieve more precise quantization. This process is expressed as $x = concatenate(x_1, x_2, \ldots, x_P)$, where P is the number of subvectors. Each subvector x_i is assigned its own codebook, $\{C_1, C_2, \ldots, C_P\}$, and quantized independently, as described in Eq. (13). By concatenating the VQ results from each subspace, the final quantized vector is obtained, as shown in Eq. (14). PVQ significantly expands the representation space to $O(C^P)$ by combining the codes from different subspaces, where C represents the number of codes in each codebook.

$$z_i = \underset{j}{argmax}\, x_i \cdot C_i[z_j] \tag{13}$$

$$x \approx concatenate(C_1[z_1], C_2[z_2], ..., C_P[z_P]) \tag{14}$$

4 HARDWARE DESIGN

In this section, we introduce the hardware design of VQT-CiM, including overall architecture in Sec. 4.1, hardware implementation of inner-product part in Sec. 4.2 and weighted-sum part in Sec. 4.3.

4.1 VQT-CiM Overview

Fig. 4 illustrates the overall architecture of VQT-CiM, which consists of two main parts, the FeFET arrays and the peripheral circuits. The FeFET arrays contain CiM crossbars, which perform static VMM for transformer calculations. The peripheral circuits comprise the ALU, Max Unit, Select Unit, and Reshape Unit for additional operations required by VQ, along with the LUT and Buffer for data storage, and the Softmax Unit for attention normalization. The Buffer stores the D_q and Δ_k, which are dynamically generated to represent the similarities and VQ results, respectively. The LUT holds pre-calculated similarity scores between codes from different codebooks, participating in the RVQ process. The dataflows of inner-product and weighted-sum in self-attention are depicted in the black solid arrow and red dash-dot arrow in Fig. 4, respectively.

4.2 Inner-Product Design

The hardware implementation of the inner-product part in self-attention is detailed in Fig. 5, with both RVQ and PVQ employed to enhance the model's representation capacity. For simplicity, the parameters R in RVQ and P in PVQ are both set to 2.

In the PVQ process, the inputs k_i and q_i are divided into two segments and sent to CiM_{k0} and CiM_{k1} for separate computations, as described in Fig. 5(a). To mitigate the significant latency in RVQ, which relies on sequential residual calculations and multi-stage quantization, we reformulate RVQ process as expressed in Eq. (15). The second term in this formulation, representing the similarity between codes from different codebooks, is pre-calculated and stored in the LUT, with s_{jk} denoted for similarity scores between the codebook j and k. This approach enables parallel processing with the different codebooks in RVQ, specifically C_{k0} and C_{k1} in Fig. 5(a).

$$\begin{aligned} z_k &= \underset{i}{argmax}\, x \cdot C_k[i] - (\sum_{j=1}^{k-1} C_j[z_j]) \cdot C_k[i] \\ &= \underset{i}{argmax}\, x \cdot C_k[i] - (\sum_{j=1}^{k-1} s_{jk}[z_j][i]) \end{aligned} \tag{15}$$

Fig. 5(b) illustrates the peripheral operations required for VQ. The maximum operation is utilized in RVQ_0 and RVQ_1 to identify the most similar vector for representation. In the residual layer RVQ_1, as specified in Eq. (15), the similarity scores D_{k1} are adjusted by subtracting the pre-calculated value $s_k[m]$ from the LUT, before applying the maximum operation. Since the attention score $A[i][j]$ cannot be determined until both q_i and k_j have participated in the static VMM with the codebook, the i-th row and column of the attention score matrix are obtained during the i-th cycle, by selecting the corresponding value in D_q, as described in Eq. (16). As a result, D_q, along with the maximum positions m and n, need to be stored in the buffer for future updates of attention scores.

$$\begin{cases} A[i,j] = D_{q0}[i][m[j]] + D_{q1}[i][n[j]], & j \le i \\ A[j,i] = D_{q0}[j][m[i]] + D_{q1}[j][n[i]], & j \le i \end{cases} \tag{16}$$

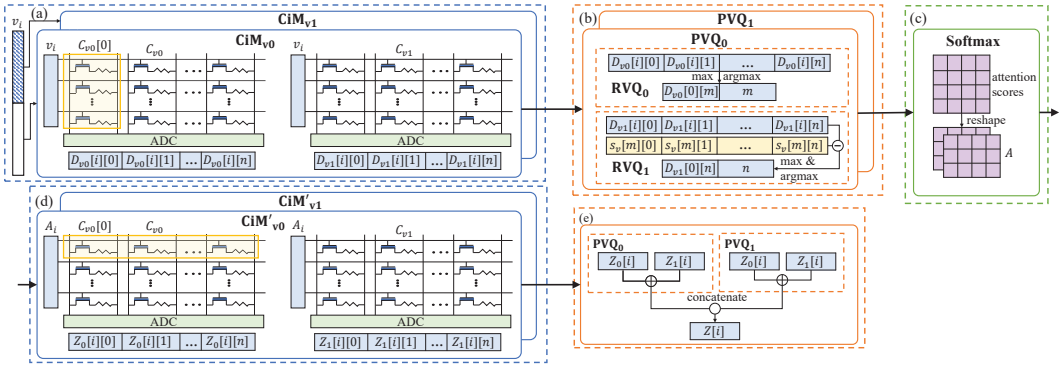

Figure 6: Hardware implementation of weighted-sum part in self-attention. (a) Static VMM with the codebook stored in FeFET crossbars. (b) Peripheral operations required by VQ. (c) Reshape the attention scores. (d) Static VMM with the codebook for weighted-sum. (e) Summation for RVQ and concatenation for PVQ.

In Fig. 5(c), the partial attention scores from different PVQ subspaces are summed and then executed by softmax unit for normalization. Apart from the conventional static VMM in the inner-product, the peripheral operations introduced by VQ lead to minimal hardware overhead. The additional operations, including maximum selection, subtraction, and summation, can be easily implemented by digital circuits. Furthermore, the LUT and buffer incur the negligible area overhead.

4.3 Weighted-Sum Design

Fig. 6 illustrates the hardware design of the weighted-sum part in self-attention, utilizing both RVQ and PVQ. The Value-VQ process, depicted in Fig. 6(a) and (b), is similar to the Key-VQ process described in Sec. 4.2, which involves splitting the input vector for PVQ, performing static VMM in the FeFET crossbar with the codebooks in RVQ, and applying peripheral operations. In Fig. 6(c), after selecting the most representative vector for value from the codebook, the attention scores obtained from the inner-product part are reshaped, corresponding to the role of Δ_v in Eq. (10). Since Δ_v varies across different subspaces in PVQ, the attention scores are replicated and reshaped accordingly, and then processed separately in Fig. 6(d) through CiM'_{v0} and CiM'_{v1} for static VMM in weighted-sum. Fig. 6(e) details the additional operations performed after obtaining the weighted-sum results, including the summation across different VQ layers in RVQ and the concatenation of different subspaces in PVQ.

Consistent with the analysis in Sec. 4.2, the additional operations introduced by the Value-VQ process, such as summation, concatenation, and reshaping, add minimal hardware overhead.

5 EVALUATION

In this section, we evaluate the proposed VQT-CiM at both algorithm and hardware level. The experimental setup is illustrated in Sec. 5.1, followed by performance analysis for algorithm and hardware in Sec. 5.2 and Sec. 5.3, respectively.

5.1 Experimental Setup

At the algorithm level, we evaluate VQT-CiM on BERT [3] with five datasets, including GLUE [31] (MRPC, QNLI), SQUAD-1.1 [32], SQUAD-2.0 [33] and IMDb [34]. We fine-tune the pre-trained BERT-base model from the Huggingface library [35]. The default sequence lengths are set to 128 for GLUE, 384 for SQUAD, and 512 for IMDb,

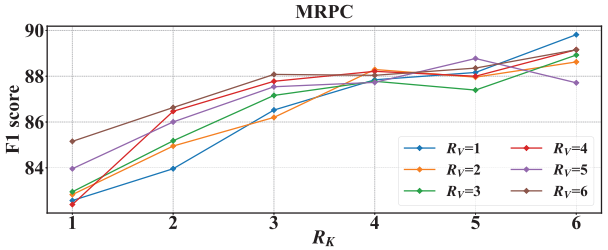

Figure 7: Model performance on MRPC under different configurations for RVQ.

with a learning rate of 2e-5 and a batch size of 32. For VQ implementations, we employ STE to address the non-differentiability in gradient backpropagation. The codebooks are initialized with k-means clustering and updated using EMA with a decay rate of 0.99. We quantize the linear transformation, inner-product, and weighted-sum in self-attention to 8 bits, employing quantization-aware training in PyTorch to maintain model performance.

At the hardware level, we evaluate the FeFET-based CiM implementations through NeuroSim [36]. VQT-CiM converts the dynamic VMM in self-attention into static VMM through VQ, making it highly compatible with NeuroSim. We set the subarray size to 32×32 and the technology node to 32nm. The read latency and ON/OFF resistances are obtained from the simulation results of Preisach FeFET model [28] through Cadence Spectre. The softmax operations are implemented using the approximation methods in I-ViT [37] and block-wise computation in SOLE [38], along with the digital peripheral circuits introduced by VQ. We estimate the energy and latency of these digital circuits with Synopsys Design Compiler through the 40nm library of UMC.

5.2 Algorithm Performance

Impact of VQ configurations. We investigate the effect of VQ configurations on model performance, focusing on the number of VQ layers in RVQ. Specifically, we explore R_K and R_V in the Key-VQ and Value-VQ processes, while keeping the codebook size C in each layer at 32 and the number of subvectors in PVQ at 2. Fig. 7 illustrates the F1 scores for the MRPC task across various configurations. The results indicate that the model performance on MRPC is more influenced by R_K than by R_V. When R_K is below 3, increasing either R_K or R_V enhances performance, with R_K having

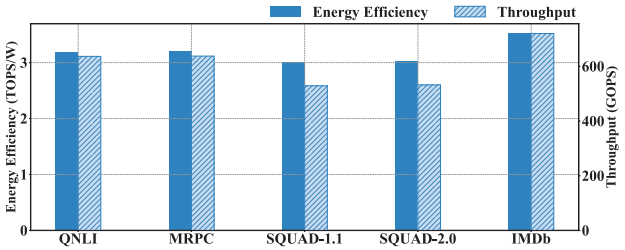

Figure 8: Energy efficiency and throughput of VQT-CiM.

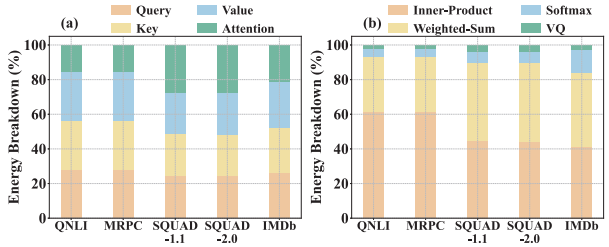

Figure 9: Energy breakdown of VQT-CiM. (a) Multi-head self-attention, including feature transformation and attention mechanism. (b) Attention mechanism, including inner-product, weighted-sum, softmax, and VQ.

a more pronounced impact. When R_K exceeds 3, VQT-CiM achieves comparable performance with the baseline, while further increasing R_K and R_V may introduce quantization noise that degrades performance. Considering the trade-off between model performance and hardware overhead, we set R_K to 4 and R_V to 2 for MRPC, leading to a 0.6% decrease in the F1 score.

Algorithm Performance Comparison. The selected configurations for various tasks and their corresponding algorithm performances are summarized in Tab. 1. For GLUE tasks with short sequence length, we set R_K to 4 and R_V to 2. For SQUAD and IMDb tasks, which involve longer sequences, we choose R_K/R_V values of 6/6 for SQUAD-1.1 and SQUAD-2.0, and 4/4 for IMDb. Under these configurations, VQT-CiM achieves comparable performance to the baseline, with an average accuracy drop of 0.8%. The quantization in feature transformation, inner-product, and weighted-sum results in a negligible average accuracy drop of 0.3%, compared to the floating-point version.

5.3 Hardware Evaluation

Hardware Performance Comparison. Fig. 8 illustrates the energy efficiency and throughput of VQT-CiM across various datasets, which include feature transformation, inner-product, weighted-sum, and softmax in multi-head self-attention. For fair comparisons, the performance metrics of ReBERT [18], ReTransformer [17], and CPSAA [19] are obtained from their corresponding NeuroSim implementations [39]. As listed in Tab. 2, VQT-CiM achieves an average energy efficiency of 3.19 TOPs/W and a throughput of 611.1 GOPs across five datasets, demonstrating improvements of 3.54×, 13.86×, 14.49× in energy efficiency, and 4.53×, 5.64×, 5.01× in throughput, compared to ReBERT, ReTransformer and CPSAA, respectively. VQT-CiM outperforms state-of-the-art NVM-based CiM designs primarily due to the elimination of runtime write operations. By removing the high overhead of write operations and

Table 1: Model performance of VQT-CiM and comparison with the baseline

Dataset	QNLI	MRPC	SQUAD-1.1	SQUAD-2.0	IMDb
Average Length	50.1	52.2	174.9	178.7	271.2
Baseline	90.7	88.9	87.8	77.0	93.0
VQT-CiM	90.4	88.3	86.4	76.0	92.4
Quantized VQT-CiM	89.8	86.8	86.2	76.6	92.5
R_K/R_V	4/2	4/2	6/6	6/6	4/4
Average Codes	96	96	192	192	128

Table 2: Energy efficiency and throughput comparison with state-of-the-art NVM-based CiM transformer designs

	ReTransformer	ReBERT	CPSAA	VQT-CiM
Technology	32nm	32nm	32nm	32nm
Implementation	ReRAM-CiM	ReRAM-CiM	ReRAM-CiM	FeFET-CiM
Energy Efficiency (TOPS/W)	0.23	0.90	0.22	3.19
Throughput (GOPs)	108.16	134.93	121.92	611.1

alleviating the complicated CWC dependencies, VQT-CiM reduces both energy efficiency and latency.

Energy Breakdown. Fig. 9(a) demonstrates the energy breakdown of multi-head self-attention in VQT-CiM, with energy consumption divided into query, key, value and attention. Different from the quadratic complexity in vanilla transformer, VQT-CiM achieves linear complexity through VQ, making the energy consumption of the attention part proportional to codebook size. Align with the number of VQ layers in Tab. 1, the attention part accounts for 15.4%, 27.2%, 21.2% of the total energy consumption in GLUE, SQUAD, IMDb, respectively. We further analyze the attention part in Fig. 9(b), breaking it down into inner-product, weighted-sum, softmax and VQ. Since the total number of codes does not scale proportionally with the sequence length in SQUAD and IMDb, the energy consumption ratio for inner-product and weighted-sum is reduced. Consequently, softmax part occupies a larger proportion of overall energy consumption, ranging from 4.7% in GLUE to 13.6% in IMDb. The VQ part, which primarily involves simple arithmetic operations such as addition and subtraction, consumes a relatively small portion of the total energy, ranging from 2.1% to 3.8%.

6 CONCLUSION

In this paper, we introduce VQT-CiM, a FeFET-based CiM design for transformer, leveraging VQ to address the challenges associated with dynamic VMM in self-attention mechanisms. We utilize Key-VQ and Value-VQ to convert the dynamic VMM in inner-product and weighted-sum into static VMM with pretrained codebooks, while incurring negligible hardware overheads. To maintain the model performance, we integrate RVQ and PVQ to significantly enhance the representation capacity of the codebooks and minimize the discrepancy between the input vector and its quantized result. The dataflow in RVQ is optimized for parallelization. Experiment results show that VQT-CiM achieves 3.54× and 4.53× improvements in energy efficiency and throughput, respectively, compared to state-of-the-art NVM-based CiM transformer accelerators.

ACKNOWLEDGMENT

This work was partially supported by Zhejiang Provincial Key R&D Program (2025C01071), NSFC (92164203) and SGC Cooperation Project (Grant No. M-0612).

REFERENCES

[1] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[2] A. Radford *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[3] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[4] A. Dosovitskiy, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.

[5] S. Yin *et al.*, "High-throughput in-memory computing for binary deep neural networks with monolithically integrated rram and 90-nm cmos," *IEEE Transactions on Electron Devices*, vol. 67, no. 10, pp. 4185–4192, 2020.

[6] X. S. Hu *et al.*, "In-memory computing with associative memories: a cross-layer perspective," in *2021 IEEE IEDM*, 2021, pp. 25–2.

[7] A. Biswas *et al.*, "Conv-ram: An energy-efficient sram with embedded convolution computation for low-power cnn-based machine learning applications," in *2018 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 2018, pp. 488–490.

[8] X. Si *et al.*, "A twin-8t sram computation-in-memory unit-macro for multibit cnn-based ai edge processors," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 1, pp. 189–202, 2019.

[9] F. Tu *et al.*, "Trancim: Full-digital bitline-transpose cim-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 6, pp. 1798–1809, 2022.

[10] F. Tu *et al.*, "Multcim: Digital computing-in-memory-based multimodal transformer accelerator with attention-token-bit hybrid sparsity," *IEEE Journal of Solid-State Circuits*, 2023.

[11] S. Liu *et al.*, "Hardsea: Hybrid analog-reram clustering and digital-sram in-memory computing accelerator for dynamic sparse self-attention in transformer," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.

[12] R. Gray, "Vector quantization," *IEEE Assp Magazine*, vol. 1, no. 2, pp. 4–29, 1984.

[13] A. Van Den Oord *et al.*, "Neural discrete representation learning," *Advances in neural information processing systems*, vol. 30, 2017.

[14] L. D. Lingle, "Transformer-vq: Linear-time transformers via vector quantization," *arXiv preprint arXiv:2309.16354*, 2023.

[15] N. Zeghidour *et al.*, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.

[16] P. Stock *et al.*, "And the bit goes down: Revisiting the quantization of neural networks," *arXiv preprint arXiv:1907.05686*, 2019.

[17] X. Yang *et al.*, "Retransformer: Reram-based processing-in-memory architecture for transformer acceleration," in *Proceedings of the 39th International Conference on Computer-Aided Design*, 2020, pp. 1–9.

[18] M. Kang *et al.*, "A framework for accelerating transformer-based language model on reram-based architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 9, pp. 3026–3039, 2021.

[19] H. Li *et al.*, "Cpsaa: Accelerating sparse attention using crossbar-based processing-in-memory architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.

[20] Y. Bengio *et al.*, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.

[21] A. Razavi *et al.*, "Generating diverse high-fidelity images with vq-vae-2," *Advances in neural information processing systems*, vol. 32, 2019.

[22] X. Yin *et al.*, "A homogeneous fefet-based time-domain compute-in-memory fabric for matrix-vector multiplication and associative search," *IEEE TCAD*, 2024.

[23] K. Ni *et al.*, "Ferroelectric ternary content-addressable memory for one-shot learning," *Nature Electronics*, vol. 2, no. 11, pp. 521–529, 2019.

[24] X. Yin *et al.*, "Deep random forest with ferroelectric analog content addressable memory," *Science Advances*, vol. 10, p. eadk8471, 2024.

[25] H. Xu *et al.*, "On the challenges and design mitigations of single transistor ferroelectric content addressable memory," *IEEE EDL*, 2023.

[26] T. Soliman *et al.*, "Ultra-low power flexible precision fefet based analog in-memory computing," in *2020 IEEE International Electron Devices Meeting (IEDM)*. IEEE, 2020, pp. 29–2.

[27] X. Yin *et al.*, "An ultracompact single-ferroelectric field-effect transistor binary and multibit associative search engine," *Advanced Intelligent Systems*, vol. 5, no. 7, p. 2200428, 2023.

[28] K. Ni *et al.*, "A circuit compatible accurate compact model for ferroelectric-fets," in *2018 IEEE symposium on VLSI technology*. IEEE, 2018, pp. 131–132.

[29] Z. Yang *et al.*, "Energy efficient dual designs of fefet-based analog in-memory computing with inherent shift-add capability," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[30] X. Yin *et al.*, "Ferroelectric compute-in-memory annealer for combinatorial optimization problems," *Nature Communications*, vol. 15, no. 1, p. 2419, 2024.

[31] A. Wang, "Glue: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.

[32] P. Rajpurkar, "Squad: 100,000+ questions for machine comprehension of text," *arXiv preprint arXiv:1606.05250*, 2016.

[33] P. Rajpurkar *et al.*, "Know what you don't know: Unanswerable questions for squad," *arXiv preprint arXiv:1806.03822*, 2018.

[34] A. Maas *et al.*, "Learning word vectors for sentiment analysis," in *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*, 2011, pp. 142–150.

[35] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.

[36] X. Peng *et al.*, "Dnn+ neurosim: An end-to-end benchmarking framework for compute-in-memory accelerators with versatile device technologies," in *2019 IEEE international electron devices meeting (IEDM)*. IEEE, 2019, pp. 32–5.

[37] Z. Li *et al.*, "I-vit: Integer-only quantization for efficient vision transformer inference," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17 065–17 075.

[38] W. Wang *et al.*, "Sole: Hardware-software co-design of softmax and layernorm for efficient transformer inference," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.

[39] X. Yu *et al.*, "Aesha: Accelerating eigen-decomposition-based sparse transformer with hybrid rram-sram architecture," in *2024 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2024, pp. 1–9.